

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2-DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I R . M A N O J K U M A R N A I K (1 9 2 5 2 4 4 3 6) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**MARKS ALLOCATION**

	Identification of Errors / Bugs(20%)	Debugging Approach & Analysis(20%)	Correctness of Debugged Solution(25%)	Optimization Techniques Applied(20%)	Testing and Documentation(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Q1. Debugging 2's Complement Overflow

Error Identification:

- Adding +120 and +50 in 8-bit 2's complement exceeds the representable range (-128 to +127).
- Binary:
 - +120 = **01111000**
 - +50 = **00110010**
 - Sum = **10101010** (incorrect because overflow occurred).

Debugging Analysis:

- Overflow occurs when two positive numbers produce a negative result.
- Carry into the sign bit is different from carry out of the sign bit.

Correct Debugged Solution:

- Detect overflow using:
 - **Overflow = Carry into MSB $\oplus$ Carry out of MSB**
- If overflow is detected:
 - Display an overflow message.
 - Use a larger data type (16-bit/32-bit) for the calculation.

Optimization:

- Implement a hardware overflow flag.
- Perform automatic range checking before arithmetic operations.

Testing & Documentation:

- Test:
 - $120 + 50 \rightarrow$ Overflow detected.
 - $30 + 20 \rightarrow 50$ (Correct)
 - $-70 + -80 \rightarrow$ Overflow detected.
- Document the overflow detection logic and expected outputs.

Q2. Ripple Carry Adder Performance

Error Identification:

- Ripple Carry Adder has high delay because each carry must propagate through every bit sequentially.

Debugging Analysis:

- Total delay increases with the number of bits.
- In real-time systems this slows arithmetic execution.

Correct Debugged Solution:

- Replace Ripple Carry Adder with **Carry Look-Ahead Adder (CLA)**.
- CLA computes carry signals in parallel using Generate (G) and Propagate (P).

Optimization:

- Parallel carry generation reduces propagation delay.
- Improves processor speed and overall throughput.

Testing & Documentation:

- Compare RCA and CLA for:
 - 8-bit addition
 - 16-bit addition
 - 32-bit addition
- Record execution time and verify identical outputs.

Q3. Booth's Multiplication Algorithm

Error Identification:

- Incorrect multiplication occurs due to:
 - Wrong bit-pair checking.
 - Improper arithmetic right shift.
 - Incorrect sign extension.

Debugging Analysis:

- Verify every Booth iteration:
 - Check Q_0 and Q_{-1} .

- Perform Add/Subtract correctly.
- Apply arithmetic shift while preserving the sign bit.

Correct Debugged Solution:

- Correct sign extension after every arithmetic shift.
- Ensure proper updates of accumulator (A), multiplier (Q), and Q_{-1} .

Optimization:

- Skip unnecessary additions for consecutive 0s or 1s.
- Improves multiplication efficiency.

Testing & Documentation:

- Test:
 - $5 \times (-3)$
 - $(-7) \times (-2)$
 - $(-8) \times 6$
- Verify results with manual calculations.

Q4. Restoring vs Non-Restoring Division

Error Identification:

- Restoring Division repeatedly restores the previous remainder after unsuccessful subtraction.
- This increases execution time.

Debugging Analysis:

- Restoration step is the main performance bottleneck.

Correct Debugged Solution:

- Replace with **Non-Restoring Division**.
- Instead of restoring immediately:
 - Perform addition or subtraction in the next cycle based on the sign of the remainder.

Optimization:

- Eliminates unnecessary restore operations.
- Reduces clock cycles.
- Improves processor efficiency.

Testing & Documentation:

- Test:
 - $25 \div 5$
 - $30 \div 6$
 - $15 \div 4$
- Compare execution time with Restoring Division and verify quotient and remainder.

Q5. IEEE 754 Floating Point Addition

Error Identification:

- Inaccurate results occur when adding very small and very large floating-point numbers.
- Causes:
 - Exponent alignment.
 - Normalization.
 - Rounding errors.

Debugging Analysis:

- During exponent alignment, smaller numbers lose significant bits.
- Precision decreases due to rounding.

Correct Debugged Solution:

- Use **IEEE 754 Double Precision (64-bit)**.
- Normalize correctly after addition.
- Apply proper rounding mode.

Optimization:

- Use compensated summation (e.g., Kahan Summation).
- Perform calculations in higher precision to reduce numerical errors.

Testing & Documentation:

- Test:

- $1e20 + 1$
 - $1.5 + 2.75$
 - $0.000001 + 1000000$
- Compare single-precision and double-precision outputs and document accuracy improvements.